

Wiping the slate clean (a.k.a Idiom 2)

At times, it is a good idea to control the environment variables—while isolating problems, you want to be absolutely sure the environment is pristine.

Here is how to do it:

```
~/atal> env -i ./tst.sh &
[1] 24302
~/atal> ps eeewww -p 24302
PID TTY STAT TIME COMMAND
24302 pts/1 SN 0:00 /bin/sh ./tst.sh
~/atal>
```

Now, I am sure I am not inheriting any environment variables.

Fine tuning

Create a file with your chosen environment variables, one assignment, per line:

```
~/atal> cat tst.env
x=25
y=26
~/atal> env -i $(cat tst.env) ./tst.sh &
[2] 24618
~/atal> ps eeewww -p 24618
PID TTY STAT TIME COMMAND
24618 pts/1 SN 0:00 /bin/sh ./tst.sh x=25 y=26
~/atal>
```

It is interesting and enlightening how the above works. All shells parse the command line. During this parsing process, they also perform various operations—one of which is substitution.

Here we use process substitution to run the *cat* command. *cat* displays the contents of the *tst.env* file to its standard output. This output is captured by the shell and split into words—the *word splitting* operation.

So, in this case, every assignment is a word, and the shell processing tacks these before the command name, effectively creating those environment variables of your choice.

Oh okay, but what about any newlines in the *tst.env* file? For the word splitting operation, both newline and spaces are delimiters (as are tabs). To see for yourself:

```
~/atal> echo $(cat tst.env)
```

IFS has the default value here—space, tab, and newline—versus:

```
~/atal> OLDIFS=$IFS
~/atal> IFS=""
~/atal> echo $(cat tst.env)
~/atal> IFS=$OLDIFS
```

The first version gives everything on the same line,

while the second preserves the newlines, as we removed the newline char from IFS. The newlines no longer act as the delimiter.

Aha! So we have some fine tuning to be done.

However, what about embedded spaces in the value of our environment variables?

```
~/atal> cat >> tst.env
z="this is a variable with a space and tab"
^D
~/atal> env -i $(cat tst.env) ./tst.sh
env: is: No such file or directory
~/atal>
```

The problem is, that by the time we finished *word splitting*, the `"` (double quotation mark) is just another character—it has lost its meaning as a protector of spaces. To understand this better, I would advise going through the shell's man pages to understand the flow. A flow chart showing how a shell processes its command line would also help.

Making passes at the command line (a.k.a the eval magic; or Idiom 3)

Don's *Art Of Computer Programming* has the following joke:

Old lady on the bus: "Young boy, can you tell me how to get off at Pasadena Street?"

Young boy: "Just watch me, and get off two stops before I do."

The joke is that the boy gives a *two pass* algorithm.

To solve the above problem we need to do one more pass before we hand over the command to *env*. Here is how to do it—just stick an *eval* before *env*.

```
~/atal> eval env -i $(cat tst.env) ./tst.sh
```

The first pass pulls the assignment strings out of the file and lays these before the command. The second pass then correctly parses these assignments and executes the command—this preserves the significance of the quotes.

Open another terminal and check out the environment. You will see the correct value of *z*. Another way to check is to stick a *echo "\$z"* in *tst.sh* before the *sleep* command and see for yourself.

Linux is so environment friendly! 

By: Atul Shrinivas Khot

The author works at IBM Software Labs in Hyderabad as a senior software engineer, and has been dabbling with UNIX/Linux for the last 14 years. He is into Java/J2EE, Groovy and Ruby these days, but at times he hankers after dear-old C++ and Perl. He loves design patterns, algorithms, multi-threading and refactoring code to make it stylish. And of course, he loves vim (never miss a chance to use it) and zsh. He collects classic British mysteries (especially those green and white Penguin crime classics—penguins make life so detectable;-)